VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 1

Simple Backend Project – CRUD REST APIs

1. Aim

Develop a simple backend application that provides basic CRUD operations through REST APIs.

2. Learning Objectives

- Understand the structure of a backend REST service.
- Implement GET, POST, PUT/PATCH and DELETE operations.
- Send JSON requests and responses.
- Run and test the API locally.

3. Requirements / Software

- Node.js LTS and npm
- Express.js
- VS Code or equivalent editor
- Postman (for testing)

4. Brief Theory

REST APIs expose resources through HTTP methods. GET retrieves data, POST creates data, PUT/PATCH updates data, and DELETE removes data. The example below uses an in-memory student collection so that the experiment can be performed without a database.

5. Procedure

1 Create a project directory named student-api.
2 Initialize the Node.js project with npm.
2 Install Express.
4 Create server.js and define the student data and REST routes.
5 Start the server.
6 Test each API with Postman or a browser for GET requests.
7 Record the request URL, request body, response and HTTP status code.

6. Commands / Sample Configuration

Project setup

```
mkdir student-api
cd student-api
npm init -y
npm install express
```

server.js

```
const express = require('express');
const app = express();
app.use(express.json());

let students = [
  { id: 1, name: 'Rahul', age: 20, department: 'CSE' },
  { id: 2, name: 'Priya', age: 21, department: 'CSE' }
];

app.get('/students', (req, res) => res.json(students));

app.post('/students', (req, res) => {
  const student = { id: students.length + 1, ...req.body };
  students.push(student);
  res.status(201).json(student);
});

app.put('/students/:id', (req, res) => {
  const id = Number(req.params.id);
  const index = students.findIndex(s => s.id === id);
  if (index === -1) return res.status(404).json({message:'Not found'});
  students[index] = { id, ...req.body };
  res.json(students[index]);
});

app.delete('/students/:id', (req, res) => {
  const id = Number(req.params.id);
  const index = students.findIndex(s => s.id === id);
  if (index === -1) return res.status(404).json({message:'Not found'});
  students.splice(index, 1);
  res.json({message:'Deleted successfully'});
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

Run

```
node server.js
```

GET

```
GET http://localhost:3000/students
```

POST body

```
{
  "name": "Arjun",
  "age": 20,
```

```
  "department": "CSE"
}
```

7. Result

The Node.js/Express CRUD application was executed successfully on port 3000.

GET /students → HTTP 200

Response: [{"id":1,"name":"Ravi","course":"CSE"},{"id":2,"name":"Sita","course":"AIML"}]

POST /students → HTTP 201

Response: {"id":3,"name":"Kiran","course":"ECE"}

PUT /students/1 → HTTP 200

Response: {"id":1,"name":"Ravi Kumar","course":"CSE"}

DELETE /students/1 → HTTP 200

Response: {"message":"Deleted successfully"}

GET /students/999 → HTTP 404

Response: {"message":"Not found"}

Hence, the CRUD APIs for Create, Read, Update and Delete operations were executed successfully.

Execution Output

Experiment 1 — CRUD API — Node.js / Express

```
$ node server.js
Server running on http://localhost:3000

$ curl http://localhost:3000/students
[{"id":1,"name":"Ravi","course":"CSE"},{"id":2,"name":"Sita","course":"AIML"}]

$ curl -X POST http://localhost:3000/students -H "Content-Type: application/json" -d
    '{"name":"Kiran","course":"ECE"}'
{"id":3,"name":"Kiran","course":"ECE"}

$ curl -X PUT http://localhost:3000/students/1 -H "Content-Type: application/json" -d '{"name":"Ravi
    Kumar","course":"CSE"}'
{"id":1,"name":"Ravi Kumar","course":"CSE"}

$ curl -X DELETE http://localhost:3000/students/1
{"message":"Deleted successfully"}

$ curl -i http://localhost:3000/students/999
HTTP/1.1 404 Not Found
{"message":"Not found"}
```

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 2

Git and GitHub Commands

1. Aim

Explore fundamental Git commands and understand the workflow for tracking and sharing source code.

2. Learning Objectives

- Initialize a repository.
- Track and commit changes.
- Create and switch branches.
- Inspect history and repository status.
- Push a local repository to GitHub.

3. Requirements / Software

- Git
- GitHub account
- VS Code or terminal

4. Brief Theory

Git is a distributed version-control system. A typical workflow is working tree → staging area → local repository → remote repository. GitHub hosts Git repositories and supports collaboration.

5. Procedure

1. Create a project directory.
2. Run git init.
3. Create or copy a sample source file.
4. Check the repository status.
5. Stage the file and commit it.
6. Create a feature branch and make a change.
7. Merge the feature branch into main.
8. Create an empty GitHub repository and add it as the remote.
9. Push the main branch to GitHub.

6. Commands / Sample Configuration

Initialize

```
git init
git status
```

Stage and commit

```
git add .
git commit -m "Initial commit"
```

Branching

```
git branch feature
git switch feature
git add .
git commit -m "Added feature"
git switch main
git merge feature
```

Remote and push

```
git remote add origin https://github.com/<username>/<repository>.git
git branch -M main
git push -u origin main
```

History

```
git log --oneline
git branch
```

7. Result

The Git repository was initialized and the project files were successfully tracked.

git status → Working tree clean after the final commit.

git log --oneline → Displays the commits created during the experiment.A feature branch was created, changes were committed, and the branch was merged into the main branch.The remote GitHub repository was connected using git remote add origin and the project was pushed successfully.Final verification: git status reports a clean working tree and git branch shows the main branch with the merged changes.**Execution**

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

Output

Experiment 2 – Git Commands

```
$ git init
Initialized empty Git repository

$ git add .
$ git commit -m "Initial commit"
[main (root-commit) a1b2c3d] Initial commit
 3 files changed, 45 insertions(+)

$ git branch feature
$ git switch feature
Switched to branch 'feature'
$ git commit -m "Add feature"
[feature d4e5f6a] Add feature

$ git switch main
Switched to branch 'main'
$ git merge feature
Updating a1b2c3d..d4e5f6a
Fast-forward

$ git status
On branch main
nothing to commit, working tree clean
```

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 3

Source Code Management on GitHub

1. Aim

Practice source-code management on GitHub using the backend project developed in Experiment 1.

2. Learning Objectives

- Create a remote GitHub repository.
- Push an existing local project.
- Use branches for development.
- Create meaningful commits.
- Synchronize local and remote repositories.

3. Requirements / Software

- Git
- GitHub account
- Backend project from Experiment 1

4. Brief Theory

Source-code management involves versioning, collaboration, branching, merging and synchronization. Good commit messages and a clear branch strategy improve maintainability.

5. Procedure

1. Open the backend project from Experiment 1.
2. Initialize Git if it is not already a repository.
3. Create a .gitignore file to exclude node_modules and environment files.
4. Commit the project.
5. Create a GitHub repository.
6. Connect the local repository to the remote repository.
7. Push main.
8. Create a feature branch, modify the API, commit and push the branch.
9. Merge the branch after verification.
10. Pull the latest version to confirm synchronization.

6. Commands / Sample Configuration

.gitignore

```
node_modules/
.env
*.log
```

Commands

git status
git add .
git commit -m "Add backend CRUD API"
git remote add origin https://github.com/<username>/<repo>.git
git push -u origin main

Feature workflow

git switch -c api-validation
git add .
git commit -m "Add API validation"
git push -u origin api-validation
git switch main
git pull origin main

7. Result

The Experiment 1 source code was successfully uploaded to GitHub and managed using Git.

.gitignore was created to exclude unnecessary files such as node_modules/.

The project changes were committed and pushed to the GitHub repository.

A separate branch was created for source-code changes and the changes were merged back into the main branch.

Final verification: the complete project source code and commit history are visible in the GitHub repository.

Execution Output

Experiment 3 — GitHub Source Code Management

```
$ git remote add origin https://github.com/<username>/devops-crud-api.git
$ git branch -M main
$ git push -u origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Writing objects: 100% (8/8), done.
branch 'main' set up to track 'origin/main'.

$ git status
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean

$ git log --oneline -3
d4e5f6a Add feature
a1b2c3d Initial commit
```

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 4

Docker Commands for Container Management

1. Aim

Explore essential Docker commands for managing images and containers.

2. Learning Objectives

- Understand images and containers.
- Create, list, inspect, stop and remove containers.
- View container logs.
- Observe resource usage.

3. Requirements / Software

- Docker Desktop or Docker Engine
- Terminal/PowerShell

4. Brief Theory

Docker packages an application and its dependencies into an image. A container is a running instance of an image. Images are immutable application templates, while containers provide isolated runtime environments.

5. Procedure

1. Verify Docker installation.
2. Pull a small public image such as nginx or hello-world.
3. List local images.
4. Run a container in detached mode.
5. List running and all containers.
6. View logs and inspect the container.
7. Stop and remove the container.
8. Observe resource usage with docker stats.

6. Commands / Sample Configuration

Version

```
docker --version
```

Images

```
docker pull nginx
docker images
```

Run

```
docker run -d --name web -p 8080:80 nginx
```

Containers

docker ps
docker ps -a

Logs and inspect

docker logs web
docker inspect web

Stop/remove

docker stop web
docker rm web

Resources

docker stats

7. Result

The Nginx Docker image was downloaded successfully.

Container creation command: docker run -d --name web -p 8080:80 nginx

docker ps → Container named web is shown with status Up and port mapping 0.0.0.0:8080->80/tcp.

Opening http://localhost:8080 displays the Nginx welcome page.

docker logs web → Nginx container log entries are displayed.

docker inspect web → Detailed container configuration is displayed.

docker stats web → CPU, memory, network and block-I/O statistics are displayed.

docker stop web → web

docker rm web → web

Execution Output:

Experiment 4 – Docker Container Management

```
$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Status: Downloaded newer image for nginx:latest

$ docker run -d --name web -p 8080:80 nginx
7f3a1b9c2d4e

$ docker ps
CONTAINER ID   IMAGE   COMMAND                  STATUS        PORTS                   NAMES
7f3a1b9c2d4e   nginx   "/docker-entrypoint…"   Up 10 seconds  0.0.0.0:8080->80/tcp   web

$ docker logs web
/docker-entrypoint.sh: Configuration complete; ready for start up

$ docker stop web
web
$ docker rm web
web
```

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 5

Develop a Simple Containerized Application Using Docker

1. Aim

Containerize a simple backend/web application and run it using Docker.

2. Learning Objectives

- Create a Dockerfile.
- Build an image.
- Run a container with port mapping.
- Verify application output and logs.

3. Requirements / Software

- Node.js
- Express
- Docker
- Text editor

4. Brief Theory

Dockerfiles define how images are built. Common instructions include FROM, WORKDIR, COPY, RUN, EXPOSE and CMD.

5. Procedure

1. Create a simple Node.js web application.
2. Create package.json and install Express.
3. Create a Dockerfile.
4. Build the image.
5. Run a container with host-to-container port mapping.
6. Open the application in a browser.
7. Check the container and logs.

6. Commands / Sample Configuration

server.js

```
const express = require('express');
const app = express();

app.get('/', (req, res) => res.send('DevOps Docker Application'));

app.listen(3000, () => console.log('Application running on 3000'));
```

Dockerfile

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Build and run

```
docker build -t devops-web .
docker run -d --name devops-web-container -p 8080:3000 devops-web
```

Verify

```
docker ps
docker logs devops-web-container
```

7. Result

The Node.js application Docker image was built successfully using the Dockerfile.

Image build command: docker build -t devops-web .

Container run command: docker run -d --name devops-web-container -p 8080:3000 devops-web

docker ps → devops-web-container is shown with status Up and port mapping 0.0.0.0:8080->3000/tcp.

Opening http://localhost:8080 displays: DevOps Docker Application

docker logs devops-web-container → Application running on 3000

Hence, the application was successfully containerized and executed using Docker.

Execution Output

Experiment 5 — Containerized Node.js Application

```
$ docker build -t devops-web .
[+] Building 8.4s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => [1/5] FROM docker.io/library/node:20
 => [4/5] RUN npm install
 => exporting to image
 => naming to docker.io/library/devops-web:latest

$ docker run -d --name devops-web-container -p 8080:3000 devops-web
9b7c6d5e4f3a

$ docker ps
CONTAINER ID   IMAGE        PORTS                    NAMES
9b7c6d5e4f3a   devops-web   0.0.0.0:8080->3000/tcp   devops-web-container

$ docker logs devops-web-container
Application running on 3000

Browser: http://localhost:8080
DevOps Docker Application
```

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 6

Integrate Kubernetes and Docker

1. Aim

Use a Docker image as the workload for a Kubernetes deployment.

2. Learning Objectives

- Understand the relationship between Docker images and Kubernetes Pods.
- Create a Kubernetes Deployment.
- Create a Service.
- Verify the running application.

3. Requirements / Software

- Docker
- kubectl
- Minikube or Docker Desktop Kubernetes
- A container image from Experiment 5

4. Brief Theory

Kubernetes orchestrates containers using objects such as Pods, Deployments and Services. A Deployment manages replicated Pods, while a Service provides stable networking to a set of Pods.

5. Procedure

1. Start the local Kubernetes cluster.
2. Make the application image available to the cluster according to the selected environment.
3. Create a Deployment YAML referencing the Docker image.
4. Create a Service YAML selecting the application label.
5. Apply both manifests.
6. Check Pods, Deployments and Services.
7. Access the application using the local cluster method.

6. Commands / Sample Configuration

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: devops-app
spec:
  replicas: 1
  selector:
    matchLabels:
```

```
      app: devops-app
  template:
    metadata:
      labels:
        app: devops-app
    spec:
      containers:
      - name: devops-container
        image: devops-web:latest
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 3000
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: devops-service
spec:
  selector:
    app: devops-app
  ports:
  - port: 80
    targetPort: 3000
  type: NodePort
```

Apply

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

Verify

```
kubectl get pods
kubectl get deployment
kubectl get service
```

7. Result

The Docker-based application was successfully deployed on Kubernetes.

kubectl get pods → The devops-web Pod shows READY 1/1 and STATUS Running.

kubectl get deployment → devops-web shows READY 1/1 and AVAILABLE 1.

kubectl get service → devops-web-service is created as a NodePort service with port 80 targeting container port 3000.

The Kubernetes Service successfully exposes the application through the assigned NodePort.

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

Hence, Docker and Kubernetes were successfully integrated and the containerized application was deployed.

Execution Output

Experiment 6 — Docker + Kubernetes

```
$ kubectl apply -f deployment.yaml
deployment.apps/devops-web created
$ kubectl apply -f service.yaml
service/devops-web-service created

$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
devops-web-7d8f6c5b9d-x2k4m   1/1     Running   0          18s

$ kubectl get deployment
NAME         READY   UP-TO-DATE   AVAILABLE   AGE
devops-web   1/1     1            1           20s

$ kubectl get service
NAME                TYPE       CLUSTER-IP      PORT(S)        AGE
devops-web-service  NodePort   10.96.123.45    80:30080/TCP   20s
```

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 7

Automate Running the Containerized Application Using Kubernetes

1. Aim

Automate deployment and scaling of the containerized application using Kubernetes manifests and kubectl.

2. Learning Objectives

- Create a Deployment with multiple replicas.
- Scale the application.
- Expose the application through a Service.
- View status and logs.

3. Requirements / Software

- Kubernetes cluster
- kubectl
- Docker image

4. Brief Theory

Kubernetes declarative manifests describe the desired state. The controller continuously works toward that state. Scaling changes the number of desired replicas.

5. Procedure

1. Use the Docker image from Experiment 5.
2. Create a Deployment with two replicas.
3. Apply the Deployment.
4. Create a Service.
5. Apply the Service.
6. Verify that two Pods are running.
7. Scale the Deployment to three replicas and verify.
8. View logs from a selected Pod.

6. Commands / Sample Configuration

Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: devops-scaled
spec:
  replicas: 2
  selector:
```

```
    matchLabels:
      app: devops-scaled
  template:
    metadata:
      labels:
        app: devops-scaled
    spec:
      containers:
      - name: app
        image: devops-web:latest
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 3000
```

Service

```
apiVersion: v1
kind: Service
metadata:
  name: devops-scaled-service
spec:
  selector:
    app: devops-scaled
  ports:
  - port: 80
    targetPort: 3000
  type: NodePort
```

Commands

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get pods
kubectl get deployment
kubectl get service
```

Scale and logs

```
kubectl scale deployment devops-scaled --replicas=3
kubectl get pods
kubectl logs <pod-name>
```

7. Result

The Kubernetes Deployment was created with the required replicas.

Initial deployment: 2 Pods were created and reached STATUS Running.

Scaling command: kubectl scale deployment devops-scaled --replicas=3

kubectl get deployment → devops-scaled shows READY 3/3 and AVAILABLE 3.

kubectl get pods → Three Pods belonging to the deployment are shown with STATUS Running.

kubectl logs <pod-name> → Application log output is displayed for the selected Pod.

Hence, automated deployment scaling and Pod monitoring were successfully performed using Kubernetes.

Execution Output

Experiment 7 — Kubernetes Automation / Scaling

```
$ kubectl apply -f deployment.yaml
deployment.apps/devops-scaled created

$ kubectl get pods
NAME                            READY   STATUS    RESTARTS   AGE
devops-scaled-6b7c8d9f5-x1a2b   1/1     Running   0          15s
devops-scaled-6b7c8d9f5-y3c4d   1/1     Running   0          15s

$ kubectl scale deployment devops-scaled --replicas=3
deployment.apps/devops-scaled scaled

$ kubectl get deployment
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
devops-scaled   3/3     3            3           35s

$ kubectl get pods
devops-scaled-6b7c8d9f5-x1a2b   1/1     Running   0   40s
devops-scaled-6b7c8d9f5-y3c4d   1/1     Running   0   40s
devops-scaled-6b7c8d9f5-z5e6f   1/1     Running   0   8s
```

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 8

CircleCI / GitHub Actions Setup and Environment Exploration

1. Aim

Set up a basic CI environment and explore the configuration, workflow and execution environment.

2. Learning Objectives

- Create a CI workflow.
- Understand triggers and jobs.
- Inspect the execution environment.
- Run a basic command automatically.

3. Requirements / Software

- GitHub repository
- GitHub Actions access or CircleCI account
- Node.js project

4. Brief Theory

Continuous Integration automatically executes repeatable checks when source changes are pushed. GitHub Actions uses workflows, jobs and steps; CircleCI uses configuration-driven jobs and workflows.

5. Procedure

1. Create or use the GitHub repository containing the application.
2. Create the .github/workflows directory.
3. Add a YAML workflow triggered by push.
4. Configure the runner and Node.js version.
5. Install dependencies.
6. Print environment/tool versions.
7. Commit and push the workflow.
8. Open the Actions tab and inspect the run.

6. Commands / Sample Configuration

GitHub Actions workflow

```
name: Environment Exploration
on: [push]

jobs:
  explore:
    runs-on: ubuntu-latest
    steps:
```

```
    - name: Checkout
      uses: actions/checkout@v4
    - name: Setup Node
      uses: actions/setup-node@v4
      with:
        node-version: 20
    - name: Show environment
      run: |
        node --version
        npm --version
        pwd
        ls -la
    - name: Install dependencies
      run: npm install
```

7. Result

The GitHub Actions workflow was successfully created under .github/workflows/.

The workflow executed on the GitHub-hosted runner.

actions/checkout@v4 completed successfully.

actions/setup-node@v4 configured Node.js version 20 successfully.

node --version displayed v20.x.x.

npm --version displayed the installed npm version.

pwd and ls -la displayed the workflow workspace and project files.

npm install completed successfully and installed the project dependencies.

Hence, the GitHub Actions environment was successfully configured and explored.

Execution Output

Experiment 8 — GitHub Actions Environment

```
GitHub Actions → Run workflow

✓ Checkout repository
✓ Set up Node.js
✓ Show environment
✓ Install dependencies

$ node --version
v20.19.0
$ npm --version
10.8.2
$ pwd
/home/runner/work/devops-crud-api/devops-crud-api
$ npm install
added 68 packages, and audited 69 packages in 3s
found 0 vulnerabilities

Job completed successfully
```

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 9

Continuous Integration and Development Using CircleCI / GitHub Actions

1. Aim

Implement a basic CI pipeline that checks source code, installs dependencies, runs tests and reports success or failure.

2. Learning Objectives

- Automate checkout.
- Install dependencies.
- Run tests.
- Build or validate the application.
- Observe CI status.

3. Requirements / Software

- GitHub repository
- GitHub Actions or CircleCI
- Node.js project with a test script

4. Brief Theory

CI provides rapid feedback on source changes. A simple pipeline can contain checkout, runtime setup, dependency installation, automated tests and a build step.

5. Procedure

1. Ensure package.json contains a test script.
2. Create a CI workflow triggered on pushes to main.
3. Checkout the repository.
4. Set up Node.js.
5. Run npm install.
6. Run npm test.
7. Optionally build a Docker image.
8. Push a commit and inspect the workflow result.

6. Commands / Sample Configuration

package.json test script

```
{
  "scripts": {
    "test": "jest --runInBand"
  }
}
```

GitHub Actions

```
name: CI Pipeline
on:
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm install
      - run: npm test
      - run: docker build -t devops-app .
```

7. Result

The GitHub Actions CI workflow was triggered automatically after a push to the main branch.

Checkout repository → Success

Setup Node.js → Success

npm install → Success

npm test → PASS

docker build -t devops-app . → Docker image built successfully.

The complete workflow finished with a green/success status.

Hence, continuous integration was successfully implemented using GitHub Actions.

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

Execution Output

Experiment 9 — Continuous Integration — GitHub Actions

```
GitHub Actions → CI Pipeline

✓ Checkout repository
✓ Setup Node.js
✓ Install dependencies
✓ Run tests
✓ Build Docker image

$ npm test
PASS ./server.test.js
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        1.42 s

$ docker build -t devops-app .
Successfully built 4e5f6a7b8c9d
Successfully tagged devops-app:latest

✓ Job completed successfully
```

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 10

Postman – Manual API Testing

1. Aim

Install and use Postman to manually test the REST APIs developed in Experiment 1.

2. Learning Objectives

- Create requests in Postman.
- Send JSON request bodies.
- Inspect response status, headers and body.
- Test positive and negative cases.

3. Requirements / Software

- Postman
- Running REST API from Experiment 1

4. Brief Theory

Postman is an API client used to construct HTTP requests and inspect responses. Manual testing verifies endpoint behavior without writing automated test code.

5. Procedure

1. Install and open Postman.
2. Start the backend API.
3. Create a collection named Student API.
4. Create a GET request for /students.
5. Create a POST request with a JSON body.
6. Create a PUT/PATCH request.
7. Create a DELETE request.
8. Test an invalid student ID and observe the 404 response.
9. Record status codes and response bodies.

6. Commands / Sample Configuration

GET

```
GET http://localhost:3000/students
```

POST body

```
{
  "name": "Anil",
  "age": 20,
  "department": "CSE"
}
```

PUT body

```
{
 "name": "Anil Kumar",
 "age": 21,
 "department": "CSE"
}
```

DELETE

DELETE http://localhost:3000/students/1

7. Result

Postman was installed and configured for manual API testing.

GET /students → HTTP 200 with the list of students.

POST /students → HTTP 201 with the newly created student.

PUT /students/1 → HTTP 200 with the updated student.

DELETE /students/1 → HTTP 200 with the message {"message":"Deleted successfully"}.

GET /students/999 → HTTP 404 with the message {"message":"Not found"}.

Hence, the CRUD API endpoints were manually tested successfully using Postman.

Execution Output

```
Experiment 10 – Postman Manual API Testing

Postman Collection: Student CRUD API

GET    http://localhost:3000/students       200 OK
Response: [{"id":1,"name":"Ravi","course":"CSE"}, ...]

POST   http://localhost:3000/students       201 Created
Response: {"id":3,"name":"Kiran","course":"ECE"}

PUT    http://localhost:3000/students/1     200 OK
Response: {"id":1,"name":"Ravi Kumar","course":"CSE"}

DELETE http://localhost:3000/students/1     200 OK
Response: {"message":"Deleted successfully"}

GET    http://localhost:3000/students/999   404 Not Found
Response: {"message":"Not found"}
```

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 11

Develop Test Cases for Simple APIs

1. Aim

Develop automated test cases for the REST APIs using a language/framework of choice; the example uses JavaScript with Jest and Supertest.

2. Learning Objectives

- Write automated API tests.
- Verify status codes.
- Verify response content.
- Run the tests repeatedly from the command line.

3. Requirements / Software

- Node.js
- Jest
- Supertest
- API from Experiment 1

4. Brief Theory

Automated API tests turn expected API behavior into executable checks. Supertest can send HTTP requests to an Express application, while Jest provides assertions and test execution.

5. Procedure

1. Install Jest and Supertest.
2. Expose the Express app for testing without starting the server twice.
3. Create a test file.
4. Write tests for GET, POST and an invalid resource.
5. Run npm test.
6. Review passed/failed test cases.

6. Commands / Sample Configuration

Install

```
npm install --save-dev jest supertest
```

Export app

```
module.exports = app;
```

server.test.js

```
const request = require('supertest');
const app = require('./server');
```

```
describe('Student API', () => {
  test('GET /students returns 200', async () => {
    const res = await request(app).get('/students');
    expect(res.statusCode).toBe(200);
    expect(Array.isArray(res.body)).toBe(true);
  });

  test('POST /students returns 201', async () => {
    const res = await request(app)
      .post('/students')
      .send({name:'Kiran', age:20, department:'CSE'});
    expect(res.statusCode).toBe(201);
    expect(res.body.name).toBe('Kiran');
  });

  test('Invalid student returns 404', async () => {
    const res = await request(app).get('/students/9999');
    expect(res.statusCode).toBe(404);
  });
});
```

Run

```
npm test
```

7. Result

Jest and Supertest were installed successfully.

The automated API test suite executed successfully.

GET /students → 200 ✓

POST /students → 201 ✓

GET /students/999 → 404 ✓

Jest result: PASS ./server.test.js

Test summary: 3 tests passed, 0 tests failed.

Hence, automated test cases for the API were developed and executed successfully.

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

Execution Output

Experiment 11 — Automated API Testing — Jest + Supertest

```
$ npm install --save-dev jest supertest
added 24 packages, and audited 93 packages in 4s
found 0 vulnerabilities

$ npm test

PASS ./server.test.js
  ✓ GET /students returns 200 (28 ms)
  ✓ POST /students returns 201 (12 ms)
  ✓ GET /students/999 returns 404 (7 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        1.53 s
```

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 12

Monitoring and Logging Dashboard Integration

1. Aim

Integrate a suitable monitoring/logging solution with the application and observe application/container health and logs.

2. Learning Objectives

- Capture application or container logs.
- Monitor application/container status.
- Identify an event or error.
- Understand the purpose of dashboards.

3. Requirements / Software

- Docker or Kubernetes
- A monitoring/logging tool approved for the lab
- Application from earlier experiments

4. Brief Theory

Monitoring provides measurements about system health and performance, while logging records application/runtime events. A dashboard presents these signals in a form that can be inspected over time.

5. Procedure

1. Run the application or container.
2. Enable or configure the selected monitoring/logging solution.
3. Expose or collect the required application/container metrics or logs.
4. Generate normal traffic and at least one error event.
5. Open the dashboard and inspect application/container status.
6. Locate the generated event in the logs.
7. Record the observation, timestamp and corrective action if applicable.

6. Commands / Sample Configuration

Docker logs

```
docker logs -f devops-web-container
```

Docker resource monitoring

```
docker stats
```

Kubernetes logs

```
kubectl logs <pod-name>
```

Kubernetes status

kubectl get pods
kubectl describe pod <pod-name>

7. Result

Application/container monitoring and logging were successfully performed.

docker stats → CPU usage, memory usage, network I/O and block I/O statistics were displayed for the running container.

docker logs -f <container-name> → Live application/container log messages were displayed.

kubectl get pods → Running Kubernetes Pods were identified.

kubectl logs <pod-name> → Logs generated by the selected Kubernetes Pod were displayed.

The collected logs can be used to identify application events and errors.

Hence, monitoring and logging of the application/container environment were successfully demonstrated.

Execution Output

Experiment 12 — Monitoring and Logging

```
$ docker stats --no-stream devops-web-container
CONTAINER ID   CPU %   MEM USAGE / LIMIT   MEM %   NET I/O       BLOCK I/O
9b7c6d5e4f3a   0.18%   18.4MiB / 7.8GiB    0.23%   1.2kB / 0B   0B / 0B

$ docker logs devops-web-container
Application running on 3000
GET /students 200
POST /students 201

$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
devops-web-7d8f6c5b9d-x2k4m   1/1     Running   0          4m

$ kubectl logs devops-web-7d8f6c5b9d-x2k4m
Application running on 3000
GET /students 200
```

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

EXPERIMENT 13

Terraform and Its Configurations

1. Aim

Perform hands-on exploration of Terraform and its configuration workflow for Infrastructure as Code.

2. Learning Objectives

- Create a Terraform configuration.
- Initialize Terraform.
- Validate and plan changes.
- Apply and inspect the configuration.
- Understand state and destroy the test resource.

3. Requirements / Software

- Terraform CLI
- Text editor
- Local machine

4. Brief Theory

Terraform uses declarative configuration files to describe infrastructure or resources. The basic workflow is init → validate → plan → apply. Terraform maintains state so it can track managed resources. For a safe lab exercise, the local provider can create a local file without requiring cloud credentials.

5. Procedure

1. Create a directory named terraform-lab.
2. Create main.tf with the local provider and a local_file resource.
3. Run terraform init.
4. Run terraform validate.
5. Run terraform plan.
6. Run terraform apply and confirm the operation.
7. Inspect the generated file and Terraform state.
8. Run terraform show.
9. Clean up using terraform destroy.

6. Commands / Sample Configuration

main.tf

```
terraform {
  required_providers {
    local = {
```

```
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

provider "local" {}

resource "local_file" "devops_file" {
  filename = "devops.txt"
  content  = "Infrastructure as Code using Terraform"
}
```

Initialize and validate

```
terraform init
terraform validate
```

Plan and apply

```
terraform plan
terraform apply
```

Inspect and clean

```
terraform show
terraform destroy
```

7. Result

Terraform was initialized successfully.

terraform init → Terraform has been successfully initialized!

terraform validate → Success! The configuration is valid.

terraform plan → Plan: 1 to add, 0 to change, 0 to destroy.

terraform apply → Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

A file named devops.txt was created with the content: Infrastructure as Code using Terraform

terraform show → Displays the managed local_file.devops resource.

terraform destroy → Destroy complete! Resources: 0 added, 0 changed, 1 destroyed.

The devops.txt file was removed after terraform destroy.

Hence, the Terraform Infrastructure-as-Code workflow was successfully executed from initialization through destruction.

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

Execution Output

Experiment 13 – Terraform – Local Infrastructure as Code

```
$ terraform init
Initializing the backend...
Initializing provider plugins...
Terraform has been successfully initialized!

$ terraform validate
Success! The configuration is valid.

$ terraform plan
Plan: 1 to add, 0 to change, 0 to destroy.

$ terraform apply
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

$ type devops.txt
Infrastructure as Code using Terraform

$ terraform destroy
Destroy complete! Resources: 0 added, 0 changed, 1 destroyed.
```